

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)NameError

B)10

C)50

D)UnboundLocalError

Q2. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It creates the main function

B)It prevents all imports

C)It prevents syntax errors

D)It identifies code intended to run when the file is executed directly

Q3. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A)A B

B)B

C)A

D)Nothing

Q4. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6.0

B)6

C)36

D)NameError

Q5. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)5

B)6

C)None

D)3

Q6. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
        x = 30
```

```
    inner()
```

```
    print(x)
```

```
outer()
```

```
print(x)
```

A)20

10

B)30

20

C)30

30

D)20

30

Q7. What is the output?

```
def fun(a=10, b=20):
```

```
    return a - b
```



print(fun(30))

- A)10
- B)-10
- C)20
- D)30

Q8. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)20
- B)UnboundLocalError
- C)NameError
- D)10

Q9. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)0
- B)NameError
- C)None
- D)TypeError

Q10. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)InvalidAgeError
- 15

- B)Nothing, because custom exceptions cannot be caught
- C)InvalidAgeError
- Age is too low
- D)Exception
- Age is too low

Q11. What will be the output?

import math

print(math.sqrt(25))

- A)5
- B)25
- C)sqrt(25)
- D)5.0

Q12. What will be printed?

x = 10

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

outer()

- A)30
- B)20
- C)10
- D)NameError

Q13. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)20
- B)10
- C)None
- D)RuntimeError

Q14. What will be printed?



```
def fun():
    print("A")
    return
    print("B")
```

fun()

A)Nothing

B)A

C)B

D)A B

Q15. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

A)25

B)15

C)SyntaxError

D)TypeError

Q16. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)B

B)Nothing

C)A and B

D)A

Q17. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A)5 15

B)5 5

C)15 15

D)15 5

Q18. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded and executed twice

B)The module is normally loaded once and reused

C)SyntaxError

D)ImportError

Q19. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)10

10

C)Module loaded

Module loaded

10

D)Module loaded

10

Q20. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)First provides the module namespace; second imports sqrt directly

B)First imports only sqrt

C)There is no difference



D)Both import only sqrt

Q21. if test.py has
print("A")

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

- A)Nothing
- B)A
- C)A followed by B
- D)B

Q22. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

- A)3
- B)RecursionError
- C)6
- D)TypeError

Q23. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

- A)[10]
- [10, 10]
- B)[10, 10]
- [10]
- C)Error
- D)[10]
- [10]

Q24. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(
```

```
lambda x: x * 2,  
    filter(lambda x: x % 2 == 0,  
numbers)  
    )  
)
```

print(result)

- A)[2, 4, 6, 8, 10]
- B)[1, 3, 5]
- C)[2, 4]
- D)[4, 8]

Q25. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

print(outer())

- A)10
- B)5
- C)UnboundLocalError
- D)15

Q26. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```



A)1

1

1

1

B)1

2

1

3

C)0

1

0

2

D)1

2

3

4

Q27. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)SyntaxError

C)TypeError

D)20

Q28. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

A)A

B)SyntaxError

C)A B

D)B

Q29. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)15

B)5

C>Error

D)10

Q30. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)Handled

B)RuntimeError cannot propagate through functions

C)NameError

D)TypeError

Q31. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

A>Error

B)10

C)5

D)func(5)

Q32. Why is

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")  
codes's second handler never reached?
```

A)ZeroDivisionError occurs before Exception

B)except blocks execute randomly

C)ZeroDivisionError is not an exception

D)Exception is a superclass of ZeroDivisionError



Q33. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        return x + 5  
  
    return inner
```

```
f = outer()  
print(f())  
A)15  
B)None  
C)NameError  
D)10
```

Q34. What will happen?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal y  
        y = 20  
  
    inner()
```

```
outer()  
A)NameError  
B)20  
C)10  
D)SyntaxError
```

Q35. What will be printed?

```
from functools import reduce  
  
numbers = [1, 2, 3, 4]  
  
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)  
  
print(result)  
A)24  
B)10  
C)0  
D)9
```

Q36. What will be printed?

```
def fun(a, b):  
    return a + b  
  
print(fun(10, b=20))  
A)20  
B)SyntaxError  
C)30  
D)TypeError
```

Q37. Which object keeps track of modules that have already been imported during the current Python process?

```
A)sys.imports  
B).__module__  
C)sys.modules  
D)sys.path
```

Q38. What will be printed?

```
x = 10  
  
f = lambda: x  
  
x = 20  
  
print(f())  
A)10  
B)NameError  
C)None  
D)20
```

Q39. Suppose sample.py contains:

```
x = 10  
  
def display():  
    print("Hello")
```

What will this print?

```
import sample  
  
print(sample.x)  
sample.display()  
A)10  
Hello  
B)Only Hello
```



C)x
display
D)Error

Q40. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)None
B)Hello None
C)Error
D)Hello

Q41. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

A)SyntaxError
B)NameError
C)TypeError
D)It executes correctly

Q42. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

A)A
B
C
D
10

B)A
B
D
10
C)A
C
D
20
D)A
B
10
D

Q43. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

A)A
B)B C
C)B
D)A C

Q44. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

A)ValueError
B)Nothing
C)Wrong Value
D)e

Q45. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went wrong")
```

```
try:  
    test()  
except MyError as e:
```



```
print(type(e).__name__)
```

- A)TypeError
- B)MyError
- C)Something went wrong
- D)Exception

Q46. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B

B)A C

C)A

D)B C

Q47. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)random cannot be imported
- B)The local file can shadow the intended standard-library module
- C)Python automatically renames the local file
- D)Python will always reject it

Q48. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)No Output

B)B

C)A, followed by propagation of the same exception

D)A, and execution ends normally

Q49. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
    raise ValueError("negative")
```

```
    return x
```

```
except ValueError:
```

```
    return 100
```

```
finally:
```

```
    print("cleanup")
```

```
print(fun(-5))
```

A)cleanup

100

B)negative

cleanup

C)cleanup

-5

D)100

cleanup

Q50. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally followed by 10

B)Finally only

C)10 followed by Finally

D)10 only

